Security, Contact Form and Email Delivery Architecture

1. Security Architecture

The Heavens website uses a statically generated Astro architecture deployed through Netlify.

This design substantially reduces the traditional application attack surface because the public website does not initially contain:

- a SQL database;
- user authentication;
- account-management endpoints;
- an application server maintained by Heavens;
- server-side rendering;
- arbitrary file uploads;
- payment processing;
- administrative interfaces;
- a custom public API.

Static Site Generation removes many common risks associated with dynamic applications, including application-level SQL injection and vulnerabilities caused by custom runtime server logic.

However, the website must not be described as automatically secure or impossible to attack.

Remaining security risks include:

- vulnerable JavaScript dependencies;
- compromised Git or Netlify accounts;
- malicious third-party scripts;
- DNS or domain compromise;
- incorrect security headers;
- cross-site scripting caused by unsafe content rendering;
- contact-form spam and abuse;
- exposed secrets in the repository or build environment;
- supply-chain compromise;
- phishing and business-email compromise;
- misconfigured preview deployments.

Security must therefore be treated as a continuous production responsibility rather than as a benefit guaranteed solely by the static architecture.

---

2. Security Headers

The production website must implement and verify the following headers:

- `Content-Security-Policy`
- `Strict-Transport-Security`
- `X-Content-Type-Options`
- `Referrer-Policy`
- `Permissions-Policy`
- protection against unauthorized framing

A Content Security Policy should initially be deployed in report-only mode where practical, reviewed for violations, and then enforced after required sources are confirmed.

The CSP must explicitly account for:

- the website origin;
- Astro-generated assets;
- self-hosted fonts;
- images;
- GSAP scripts bundled through the project;
- Netlify Forms;
- analytics, only when approved;
- any future video or embedded media provider.

Inline scripts should be minimized. When unavoidable, an appropriate hash or nonce strategy should be evaluated instead of broadly allowing unsafe inline execution.

Security headers must be configured through `netlify.toml` or a published `_headers` file and verified against actual production responses after deployment. Netlify supports custom response headers and CSP configuration through these mechanisms.

---

3. Contact Form Architecture

The contact form will use Netlify Forms.

The visible form must be rendered as normal static HTML during the Astro build so Netlify can detect and register it.

The Fetch API may enhance the submission experience, but JavaScript must not replace the underlying form markup.

Recommended form identity:

```
<form
  name="premium-contact"
  method="POST"
  action="/en/contact/success/"
```

```
  data-netlify="true"
  data-netlify-honeypot="bot-field"
>
  <input type="hidden" name="form-name" value="premium-contact" />
  <input type="hidden" name="locale" value="en" />
</form>
```

Use one stable internal form name for all languages:

```
premium-contact
```

The localized page should submit its active locale through the hidden `locale` field.

The form should include:

- full name;
- company name;
- email;
- optional phone number;
- country;
- enquiry type;
- message;
- privacy acknowledgement;
- hidden locale;
- honeypot field.

Netlify requires the form markup to be present in the generated HTML for detection. Form submissions can then be monitored and routed through Netlify's forms system.

---

4. Spam Protection

The initial spam-protection strategy must include:

- Netlify's built-in spam filtering;
- a hidden honeypot field;
- reasonable maximum input lengths;
- no file attachments;
- server-side platform validation;
- monitoring of spam and verified submissions.

A completed honeypot field indicates likely automated submission and can be rejected by Netlify.

The implementation must not claim that all spam will be eliminated. False positives and undetected spam remain possible.

During testing, submissions should be checked in both:

- verified submissions;

- spam submissions.

Netlify may classify repeated or obviously artificial test submissions as spam.

If spam becomes a production problem, evaluate:

- reCAPTCHA;
- rate limiting through an additional controlled endpoint;
- stricter field validation;
- blocking repeated abusive patterns.

---

5. Company Email Address

The final Heavens email address has not yet been confirmed.

Until it is verified, use the following configuration placeholder:

```
HEAVENS_CONTACT_EMAIL
```

Do not hard-code an assumed mailbox such as:

```
info@heavens.am
```

throughout the project before confirming that it exists and is actively monitored.

Once confirmed, define the email address in one central company configuration file:

```ts
export const company = {
  legalName: "Heavens LLC",
  registrationNumber: "50456518",
  taxCode: "02660767",
  address: "40 Sayat-Nova Avenue, Yerevan, Armenia",
  contactEmail: import.meta.env.PUBLIC_HEAVENS_CONTACT_EMAIL,
} as const;
```

Alternatively, if the address is public and non-sensitive, it may be held in a typed content configuration file rather than an environment variable.

Before launch, verify:

- exact mailbox name;
- exact company domain;
- ability to receive external messages;
- ability to send replies;
- mailbox ownership;
- spam-folder behavior;

- SPF;
- DKIM;
- DMARC;
- who in the company monitors the inbox;
- expected response time.

Recommended mailbox naming options, subject to confirmation:

```
info@company-domain
contact@company-domain
office@company-domain
business@company-domain
```

The public website should use only the confirmed official address.

---

6. Netlify Form Email Notifications

After deploying the form and confirming that Netlify recognizes `premium-contact`, configure the notification in the Netlify dashboard.

Current navigation may vary slightly, but the intended setup is:

```
Project configuration
→ Notifications or Forms
→ Form notifications
→ Add notification
→ Email notification
```

Configuration:

```
Event:
New verified form submission

Form:
premium-contact

Recipient:
Confirmed Heavens domain email address
```

Netlify supports email notifications for individual forms or for all forms associated with a site.

By default, notification messages are sent by Netlify. A contact-form input named `email` should be included because Netlify can use it as the notification's reply-to address, allowing the team to reply more conveniently to the submitter.

The recipient email is configured inside Netlify and does not need to be embedded in the contact-form submission code.

However, the official Heavens email may still appear publicly on:

- Contact;
- Legal Information;
- Privacy Policy;
- footer.

This is normal for a corporate website and should not be confused with exposing private application credentials.

---

7. Submission Flow

The intended production flow is:

```
Visitor
→ HTTPS contact form
→ Netlify Forms
→ Spam evaluation
→ Verified submission storage
→ Email notification
→ Confirmed Heavens company mailbox
→ Manual business response
```

Heavens does not need to operate:

- an SMTP server;
- a custom email API;
- a custom form database;
- a serverless mail function;

for the initial version.

Netlify Forms provides the managed ingestion and notification layer. This is still a third-party data-processing service and must be disclosed in the Privacy Policy.

---

8. Data Minimization

The contact form must collect only information reasonably required to respond to a business enquiry.

Do not collect:

- identity documents;
- payment details;
- passwords;

- medical information;
- highly sensitive personal data;
- unnecessary attachments;
- confidential business files through the general contact form.

Recommended field limits:

```
Full name: 100 characters
Company name: 150 characters
Email: 254 characters
Phone: 40 characters
Country: 100 characters
Enquiry type: fixed allowed values
Message: 3,000–5,000 characters
```

All input must be validated and normalized before being displayed or processed elsewhere.

---

9. Submission Retention

Netlify stores form submissions in its dashboard unless they are deleted or handled according to the applicable platform settings and plan.

The project must not promise automatic deletion after 30 days until this capability has been confirmed in the actual Netlify account.

Before launch, determine:

- how long Netlify retains verified submissions;
- how long spam submissions are retained;
- whether configurable automatic deletion exists on the selected plan;
- whether manual deletion is sufficient;
- whether scheduled deletion through the Netlify API is required;
- whether email copies are retained separately in the company mailbox.

Netlify provides dashboard management for form submissions and APIs capable of managing submissions, but any automated deletion workflow must be explicitly designed, authenticated and tested.

Recommended initial retention decision:

```
Operational target:
Delete contact-form submissions from Netlify when no longer needed,
preferably within 30–90 days.

Business email:
Retain only as long as required for the enquiry, contractual follow-up, or
applicable legal obligations.
```

The final Privacy Policy must state the actual implemented retention period, not an aspirational period.

---

10. Email Security

Because the contact mailbox belongs to the company domain, domain email security is part of the website launch requirements.

Verify:

SPF

The domain must publish an SPF record authorizing the actual mail provider.

DKIM

Outbound messages must be cryptographically signed by the configured mail provider.

DMARC

The domain should publish a DMARC policy and receive aggregate reports where practical.

A safe rollout is:

```
p=none
```

for monitoring, followed later by:

```
p=quarantine
```

or:

```
p=reject
```

after all legitimate senders are confirmed.

Account security

The official mailbox must use:

- a unique password;
- multi-factor authentication;
- restricted recovery access;
- monitored forwarding rules;
- periodic access review.

Do not automatically forward all enquiries to personal email accounts unless this has been formally approved.

---

11. Privacy Policy Additions

The Privacy Policy must state that contact-form information may be processed through Netlify as the website hosting and form-infrastructure provider.

It should explain:

- fields collected;
- reason for collection;
- recipient within Heavens;
- Netlify's processing role;
- notification by email;
- retention period;
- possible cross-border processing;
- how users can request access, correction or deletion.

Suggested wording:

```
When you submit the contact form, the information you provide is processed
through our website hosting and form-infrastructure provider and is delivered
to the official Heavens LLC business mailbox so that we can review and
respond to your enquiry.

We retain enquiry information only for as long as reasonably necessary to
respond, manage related business communication, protect our legitimate
interests and meet applicable legal obligations.
```

This text must be finalized only after the actual provider, mailbox and retention setup are confirmed.

---

12. Production Verification Checklist

Before launch, confirm:

```
[ ] The official Heavens domain is confirmed.
[ ] The exact company email address is confirmed.
[ ] The mailbox can receive external email.
[ ] The mailbox can send replies.
[ ] SPF is valid.
[ ] DKIM is enabled.
[ ] DMARC is published.
[ ] Multi-factor authentication is enabled.
[ ] The Astro build contains a static premium-contact form.
[ ] Netlify detects premium-contact after deployment.
```

```
[ ] The hidden form-name field is submitted.
[ ] The hidden locale field is submitted.
[ ] The honeypot is present.
[ ] A normal submission appears as verified.
[ ] A honeypot submission is rejected or classified appropriately.
[ ] Email notifications reach the confirmed mailbox.
[ ] Reply-to uses the submitter’s email field correctly.
[ ] The form works with JavaScript.
[ ] The form has a fallback without JavaScript.
[ ] Error and success messages are accessible.
[ ] Input lengths are enforced.
[ ] No sensitive fields are collected.
[ ] Retention behavior is confirmed in Netlify.
[ ] The Privacy Policy states the actual retention process.
[ ] Security headers are verified in production.
[ ] CSP is tested before full enforcement.
[ ] Preview and branch deployments are protected from indexing.
```

---

13. Roadmap Updates

Phase 0 — Business verification

Add:

- confirm the exact Heavens domain;
- confirm the exact official email mailbox;
- identify the mailbox owner;
- identify the company email provider;
- verify SPF, DKIM and DMARC capability.

Phase 4 — Technical foundation

Add:

- create the static Netlify form;
- use the stable form name `premium-contact`;
- add the locale field;
- add the honeypot;
- configure field limits;
- configure security headers initially in report-only mode where appropriate.

Phase 9 — Security and production audit

Add:

- verify Netlify form detection;
- test verified and spam submissions;
- test notification delivery;
- inspect reply-to behavior;

- verify SPF, DKIM and DMARC;
- verify CSP;
- verify all response headers;
- verify no secrets exist in client bundles;
- inspect dependency audit results.

Phase 10 — Legal and retention review

Add:

- confirm actual Netlify submission retention;
- define deletion responsibilities;
- confirm mailbox retention;
- align Privacy Policy with actual implementation;
- document the procedure for data-access and deletion requests.

Phase 11 — Launch

Add:

- enable the final notification recipient;
- run a real external form test;
- reply to the test from the official company mailbox;
- verify delivery and spam placement;
- record the confirmed email address in central company configuration.